

Project Title

Multi-Objective Traffic Signal Optimization Using Multi-Agent Reinforcement Learning with ARIMA-Based Traffic Prediction in SUMO

1. INTRODUCTION

1.1 Purpose

This document specifies the complete functional and non-functional requirements for implementing a **multi-agent reinforcement learning (MARL) based traffic signal optimization system** integrated with:

- SUMO (Simulation of Urban Mobility)
- ARIMA-based short-term traffic flow prediction
- Multi-objective reward optimization (delay, fuel, emissions)

The target of this specification is an **AI software agent** that will autonomously implement the full system.

1.2 System Overview

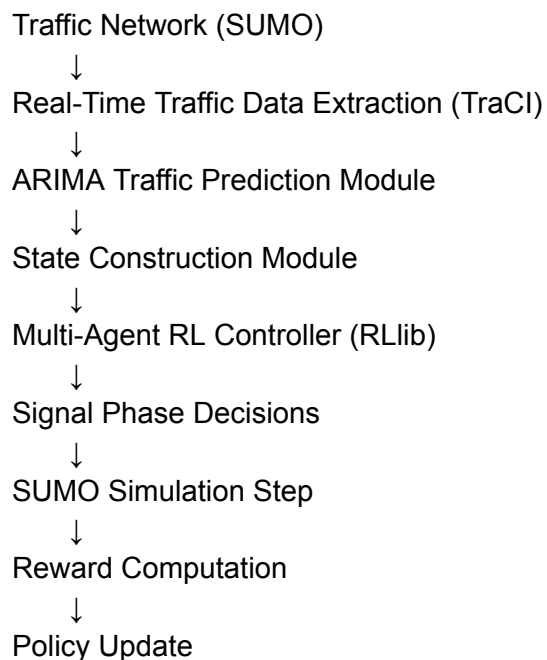
The system shall:

1. Simulate an urban traffic network in SUMO.
2. Assign one RL agent per signalized intersection.
3. Predict short-term traffic inflow using ARIMA.
4. Optimize signal timing policies via MARL.
5. Minimize:
 - Total waiting time

- CO2 emissions
 - Fuel consumption
6. Support centralized training and decentralized execution.
-

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture



3. FUNCTIONAL REQUIREMENTS

3.1 SUMO Simulation Module

3.1.1 Network Configuration

The system shall:

- Use SUMO for microscopic traffic simulation.
- Support configurable network files (.net.xml).
- Support route files (.rou.xml).
- Support traffic signal logic (.add.xml).

Required Files:

- `network.net.xml`
 - `routes.rou.xml`
 - `config.sumocfg`
-

3.1.2 Simulation Control

The system shall:

- Use TraCI for simulation control.
- Advance simulation in discrete steps.
- Allow external modification of traffic light phases.

Required Functions:

```
traci.start()
traci.simulationStep()
traci.trafficlight.setPhase()
traci.vehicle.getWaitingTime()
traci.vehicle.getCO2Emission()
traci.vehicle.getFuelConsumption()
```

3.2 Traffic Metrics Extraction

At each simulation step, the system shall compute:

3.2.1 Total Waiting Time

$$W_t = \sum_{v \in V} w(v)$$

3.2.2 Total CO2 Emission

$$C_t = \sum_{v \in V} c(v)$$

3.2.3 Total Fuel Consumption

$$F_t = \sum_{v \in V} f(v)$$

3.3 ARIMA Prediction Module

3.3.1 Purpose

Predict short-term vehicle inflow for each intersection.

3.3.2 Input

- Historical vehicle inflow time series per intersection.
-

3.3.3 Model

ARIMA(p, d, q)

Initial configuration:

- $p = 5$
- $d = 1$
- $q = 0$

3.3.4 Output

$$\hat{X}_{t+1:t+k}$$

Predicted inflow for next k steps (default k = 5).

3.3.5 Integration

Predictions must be appended to the agent state vector.

3.4 Multi-Agent RL Environment

3.4.1 Environment Type

- Gym-compatible environment.
 - Multi-agent format compatible with RLlib.
-

3.4.2 Agents

- One agent per signalized intersection.
 - Each agent has independent observation.
 - Shared policy or individual policies configurable.
-

3.4.3 State Space Definition

For each intersection:

$S = [\text{queue_length_per_lane}, \text{waiting_time_per_lane}, \text{current_phase}, \text{ARIMA_predictions}]$
 $S = [\text{queue_length_per_lane}, \text{waiting_time_per_lane}, \text{current_phase}, \text{ARIMA_predictions}]$
 $S = [\text{queue_length_per_lane}, \text{waiting_time_per_lane}, \text{current_phase}, \text{ARIMA_predictions}]$

Dimensions must be fixed and documented.

3.4.4 Action Space

Discrete action space:

Action	Description
0	Maintain current phase
1	Switch to next phase
2	Extend green duration
3	Reduce green duration

3.5 Reward Function

Multi-objective reward:

$$R = -(w_1 W_t + w_2 C_t + w_3 F_t)$$

Default weights:

- $w_1 = 0.5$
- $w_2 = 0.3$
- $w_3 = 0.2$

Weights must be configurable.

3.6 MARL Training Module

3.6.1 Framework

Use:

- Ray RLlib
- PPO (default)
- Option to extend to MADDPG

3.6.2 Training Mode

- Centralized training
- Decentralized execution

3.6.3 Configuration

- Learning rate configurable
- Discount factor configurable
- Training episodes configurable
- Batch size configurable

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance

- Must support minimum 4 intersections.

- Training time configurable.
 - Simulation must run faster than real-time where possible.
-

4.2 Scalability

- Architecture must allow scaling to N intersections.
 - Agent count must dynamically adapt to network size.
-

4.3 Modularity

Each module must be independently executable:

- SUMO Module
 - Prediction Module
 - RL Module
 - Evaluation Module
-

4.4 Logging

The system shall log:

- Episode reward
- Waiting time
- CO2
- Fuel
- Phase decisions

Logs must be stored in CSV format.

5. EXPERIMENTAL REQUIREMENTS

The system shall support the following experiment modes:

1. Fixed-time control (baseline)
2. Waiting-time optimized only
3. CO2 optimized only
4. Fuel optimized only
5. Full multi-objective MARL

Results must include:

- Percentage reduction vs baseline
- Comparative graphs

6. DATA STORAGE REQUIREMENTS

All outputs must be stored under:

```
/results/  
training_logs.csv  
evaluation_metrics.csv  
reward_plot.png  
comparison_plot.png
```

7. ERROR HANDLING REQUIREMENTS

The system must:

- Detect SUMO crash.

- Restart simulation safely.
 - Validate ARIMA training data size.
 - Ensure reward does not produce NaN.
-

8. IMPLEMENTATION CONSTRAINTS

- Python 3.10+
 - SUMO 1.15+
 - Ray RLlib
 - NumPy
 - Pandas
 - Statsmodels
-

9. VALIDATION REQUIREMENTS

The system shall validate:

- Reduction in waiting time > 10% over baseline.
 - Reduction in CO2 > 5% over baseline.
 - Reduction in fuel > 5% over baseline.
-

10. FUTURE EXTENSIONS (Optional)

- Pareto front computation

- Dynamic speed limits
 - Real traffic dataset integration
 - Deep forecasting (LSTM)
-

11. ACCEPTANCE CRITERIA

The system is considered complete if:

- Agents learn non-trivial signal policies.
- Multi-objective optimization reduces congestion and emissions.
- ARIMA improves performance over no-prediction setup.
- Full experiment results are reproducible.